Project 3 Report: An Autonomous VIO-based Quadrotor

Pranav Shah
(pranavps@seas.upenn.edu)

I. Introduction

The objective of this project is to combine project 1, which is based on trajectory generation and control of quadrotor based on the ground truth values of the position and orientation of the quadrotor, and project 2, which is based on Visual Inertial Odometry (VIO). VIO is a method to estimate the position and orientation of a quadrotor with the help of stereo images and IMU measurements.

In real world, it is impossible to get ground truth estimates of the quadrotor position, velocity and orientation and hence it is crucial to use a reliable estimator which can provide good estimates of the state of the quadrotor.

In this project, the aim is to simulate a real-world quadrotor which needs to estimate its state to be able to navigate a map. It does not have accesss to ground truth estimates of its state and hence needs to rely solely on the estimates of the VIO.

II. VIO and its Challenges

Ground truth values give us the exact position, velocity and orientation of the quadrotor and hence can be trusted with hundred percent confidence while using this information to control the quadrotor. It therefore gives us the ideal flight path and behavior of the quadrotor as designed. VIO estimates can be noisy and there is some margin of error when compared to the ground truth. VIO estimates are being estimated using an Error State Kalman Filter (ESKF).

There are several factors that can affect the VIO estimates. One of the factors is the noise covariance matrix. If the noise covariance is high, then the estimates will be very noisy with a significant margin of error and if it is too low, then the estimates will be too slow to converge. Another factor that affects the performance of the VIO system is the feature count detected among the stereo image pairs. If the feature count is too low, the confidence with which the estimates are updated is also low and hence we get faulty, noisy estimates.

The reasons due to which we may get less feature counts from our stereo images may be because of either very aggressive maneuvers, quick sharp turns or a very bad controller to correct the attitude of the quadrotor. If the trajectory has quick sharp turns or aggressive maneuvers there is a high chance that the point correspondence between consecutive stereo image pairs will have very low feature count or none at all. There will be less features matching in these corresponding pairs and hence will result in state updates with low confidence or more noise. These faulty point correspondences eventually add to the error in the estimate and lead the quadrotor astray from its desired trajectory. The solution to tackle this is to prune the trajectory well, and plan a smooth path between the waypoints so as to avoid very aggressive maneuvers. Another way to avoid aggressive maneuvers is to reduce the average speed for that segment by allowing the quadrotor more time to complete traversing a particular trajectory segment. The pruning that I performed is to consider every 7th point in the trajectory and for path planning, a minimum jerk trajectory was implemented.

There is a significant 'sim2real' difference when it comes to VIO based state estimation. In the simulation, all the features are indexed and hence feature matching becomes an easy and deterministic task. In real world, feature matching in images can result in very noisy point correspondences with several false correspondences as well. This will result in very noisy state estimates. One way to deal with this, is to be careful of the trajectory that quadrotor follows and also to avoid any aggressive maneuvers to allow for good matches among consecutive stereo images.

The VIO system is working well for some maps in the simulation because there are either walls or obstacles constantly around the quadrotor, for example like the 'maze' map. For maps like the 'window', there is a section of the map, where the quadrotor barely detects any features because of large distance from any obstacle. It is at such sections where there quadrotor cannot be made to do very aggressive maneuvers or it would lose sense of states and would either crash or drift away.

III. Results from First Solution

In the first solution, the code was simply combined from project 1 (trajectory generator and controller) and project 2 (VIO) and the avg speed was reduced to 3 ms^{-1} from 3.6 ms^{-1} used in project 1. The results of this solution are in the table below:

Map name	Flight time (s)	Flight Distance (m)
maze	8.32	23.73
over_under	11.61	33.82
window	9.38	26.04
stairwell	25.16	73.78
switchback	25.33	74.43
slalom	11.77	33.89

TABLE I

RESULTS OF FIRST SOLUTION

As evident from the table above, this solution is still is a bit slow in some maps and does not take into account any sharp turns or aggressive maneuvers in the path planning for the quadrotor.

IV. RESULTS FROM SECOND SOLUTION

This solution is built on top of the first solution and hence required very minor changes. The major difference in this solution is the time allocation for each segment of the trajectory. For example, if the angle between segment A and segment B of the trajectory is above $25°$, then the segment A is given more time to traverse so that the quadrotor is in a bit of a relaxed state while entering the turn. This time allocation is non-linear due to the use of sine and cosine of the angles between the segments. The resolution was also changed from 0.25 to 0.21. The speed was reduced to 2.9 ms^{-1} The results of this solution are in the table below:

Map name	Flight time (s)	Flight Distance (m)
maze	8.39	23.16
over_under	12.09	32.36
window	8.97	26.15
stairwell	12.3	33.65
switchback	26.11	74.02
slalom	19.84	57.49

TABLE II

RESULTS OF SECOND SOLUTION

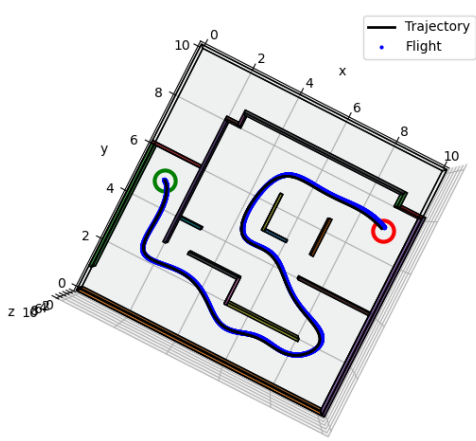

Fig. 1. Maze 3D map with trajectory

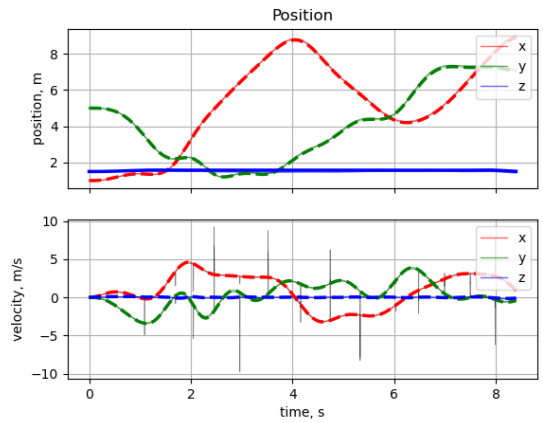

Fig. 2. Controller commands and actual outputs

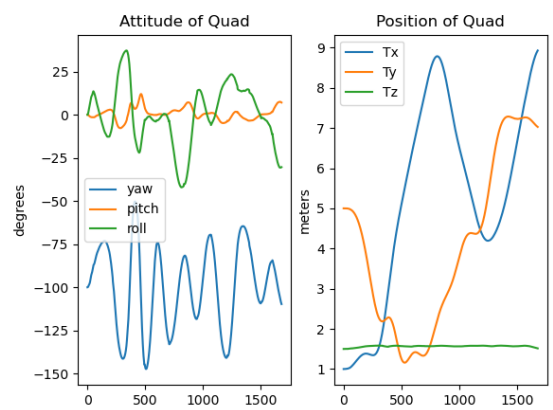

Fig. 3. VIO estimates

We can see that the position commands from the controller, the actual position and the position estimates from the VIO all match well.

V. CONCLUSION

The controller and trajectory generator seems to be doing their job well but there is still room for improvement. One of the few things that I would continue looking into is the effect of resolution and margin on the trajectory and the renewed pruning that would be required for the same. Another area to look into as well is varied average speed per segment. For consecutive segments that do not have a large angle between them, the average speed for these segments can be high compared to segments that have a large angle between them. This would help in a slow, smooth motion of the quadrotor around sharp turns/corners.